APPLE'S ON-DEVICE FOUNDATION MODEL, REBUILT: A COMPLETE FROM-WEIGHTS RECONSTRUCTION OF `AFMPLUS-V11.2-IFP` (AFM 3 CORE ADVANCED) AND A PORTABLE LLAMA.CPP RUNTIME

YU YAO-HSING

Abstract. We reconstruct Apple's current on-device foundation language model—asset `com.apple.fm.language.instruct_3b.base.generic_sparse`, display version `afmplus-v11.2-ifp`, the model Apple calls *AFM 3 Core Advanced*—entirely from the files Apple Intelligence installs on a Mac, and run it in llama.cpp on any operating system. The model is a 56-layer, 20.3-billion-parameter transformer with a 1536-wide residual stream, 16 query and 8 key/value heads, KV-cache sharing across its last 21 layers, an unusual post-normalised ("x/y") residual with four learned RMSNorm gains per layer, and an Instruction-Following-Pruned sparse feed-forward network: 23 always-on experts plus 23 of 348 routed experts per token, chosen by a separate router network evaluated once at layer 12. Every 2-bit weight is carried into GGUF *bit-exactly* (55 million spot-checked weights, zero differences); the tokenizer reproduces Apple's own token counts on 81/81 test texts. The two things the model files do not contain—the prompt template and the rule that picks experts—we read out of Apple's inference frameworks: the conversation is rendered as a system turn plus a separate developer turn, and the experts for a whole prompt are chosen from the last prompt token's hidden state and re-chosen every 32 positions. Both are confirmed by Apple's public token counter and by Apple's own runtime log. Against Apple's live model at temperature zero, the reconstruction's next-token choice agrees with Apple's on **96.1%** of answer tokens, and 17 of 40 free-running answers are identical to Apple's character for character. The last four points came from two layer-0 details found late: llama.cpp applied the embedding scale twice (Apple applies none), and Apple adds the layer-0 input bias only to generated tokens, never to the prompt. Nothing in the model is trained or fitted.

For most of this project the assembled model predicted nothing. The cause was a single misaligned read: every normalisation gain was taken one element early. We found it with probes *calibrated* on a healthy Apple model—Xcode's on-device code model, whose MPSGraph package we dumped to readable MLIR, executed on the GPU from our own process and reproduced exactly in NumPy. The report documents the model's full structure, the storage formats, the debugging instruments and their calibration, the prompt template and expert schedule recovered from Apple's runtime, the llama.cpp implementation, a standalone converter, every negative result, and what still separates the port from 100%.

Contents

1. Results at a glance 2
2. Which model is this? 2
3. Where the model lives 3
4. Architecture 3
4.1. Dimensions 3
4.2. The layer: a post-normalised "x/y" residual 3
4.3. Attention 5
4.4. The Instruction-Following-Pruned feed-forward network 5
5. How the weights are stored 5
6. The norm-gain table, and the bug that hid the model 6
7. Validation instruments and their calibration 7

Date: November 8, 2026.

8. What Apple's runtime does: the prompt and the expert schedule 8
8.1. The prompt template 9
8.2. The expert schedule 10
9. The llama.cpp implementation 10
10. Fidelity 11
11. The converter 13
12. What did not work 13
13. Toward 100% 14
14. Responsible use 14
Appendix A. GGUF layout 15

1. Results at a glance

Model	`afmplus-v11.2-ifp` = AFM 3 Core Advanced (on-device, sparse IFP)
Size	20.30 B parameters stored; 1–4 B active per prompt (ifp1/2/3 budgets)
Layers	56 = 12 dense + 23 sparse (own K/V) + 21 sparse (reuse layer-34 K/V)
Width	$d = 1536$; 16 Q / 8 KV heads × 128; dense FFN 3072; expert width 256
Weights	2-bit palette $\{-1.5, -0.5, 0.5, 1.5\}$ × fp16 row scale; int4 embedding
GGUF	6.31 GiB, Q2_K (bit-exact) + Q4_0 (exact); custom arch `afm`
Fidelity	96.1% teacher-forced top-1 vs Apple (40 prompts, 1127 tokens); 17/40 answers identical
Runtime rules	`afm_cider_v2` template and IFP expert schedule, decoded from Apple's frameworks
Tokenizer	matches Apple's `SystemLanguageModel.tokenCount` on 81/81 texts
Runtime	llama.cpp fork: Metal, CUDA, Vulkan, CPU; 150 tok/s prompt, 12–21 tok/s generation (M4)
Converter	`afm2gguf.py`: 2–3 min, reads only the local assets, self-validating
Root cause of the long failure	norm-gain table read 2 bytes (one fp16) early

Table 1. Headline numbers. All measurements in this report were taken on an M4 Mac running macOS 27.

Code: `https://github.com/yu314-coder/afm-ifp-teardown` (llama.cpp patch, converter, build workflow). No Apple model data is published; the converter rebuilds the GGUF from the assets on the user's own machine.

2. Which model is this?

Apple's third-generation foundation models (announced June 2026) comprise two on-device language models and several Private Cloud Compute models (Table 2). The macOS 27 SDK exposes the on-device model as `SystemLanguageModel` and the server models through `PrivateCloudComputeLanguageModel`. The assets on the device contain exactly one foundation language backbone, `afmplus-v11.2-ifp`; the server models are present only as adapter metadata (`afmplus-v8.0-150b`, `afmplus-v9.0-3b-pcc`).

So the reconstructed model is *AFM 3 Core Advanced*, not a "Pro" model: the Pro model runs only on Apple's servers and is not on the device.

Name	Where	Size	Architecture	On this Mac?
AFM 3 Core	device	3 B	dense	no separate asset found
AFM 3 Core Advanced	device	**20 B**, 1–4 B active	**sparse, IFP**	**yes: `afmplus-v11.2-ifp`**
AFM 3 Cloud	PCC	n/a	n/a	metadata only
AFM 3 Cloud Pro	PCC (GPUs)	n/a	n/a	no

Table 2. Apple's 2026 lineup as described in public WWDC 2026 coverage (secondary sources; Apple's own documentation does not list sizes). The match for v11.2 rests on our measurements: llama.cpp counts 20.30 B parameters in the converted model; its feed-forward network is Instruction-Following-Pruned; and the Foundation Models API ships three compute budgets (`fm_api_generic_1b/2b/3b`) that select 5, 14 or 23 routed experts—the "1–4 B active per prompt".

3. Where the model lives

Apple Intelligence downloads its models as MobileAssets under `/System/Library/AssetsV2/com_apple_MobileAsset_UAF_FM_GenerativeModels`. The model files sit in an APFS cryptex that the system grafts into the asset's `.AssetData` directory.

File	What it holds
`binary_0.hwx` (694 MB) in `main-h16g-delegates/MPSGraph/`	The compiled ANE program: 108 procedures (prompt / extend / dense-only / sparse-only / readout, per sequence length and IFP budget). Segment `__KERN_3` holds every attention and dense-FFN weight; `__TEXT,__const` holds the norm gains; the task stream holds the int8 K/V calibration.
`main-h16g.odix` (263 MB)	The orchestration program (ODIE IR). Holds the int4 token embedding (text, audio) and its per-token fp16 scales, and the RoPE tables.
`ifp/ifp_rasterized_weights.bin` (4.9 GB)	All 44×380 experts: 2-bit codes, gate/up and down row scales.
`ifp/project_experts.mlasset`	The IFP router network (fp16).
`ifp/config.json`	IFP budgets: ifp1/2/3 = 5/14/23 active experts; `expert_selection_frequency` = 32.
`tokenizer` (separate asset)	328 192-entry SentencePiece-BPE vocabulary in a flat Apple format (262 144 text tokens + audio/voice tokens).

Table 3. Files used by the reconstruction.

Reading Apple's graphs. A private method on `MPSGraphExecutable`, `-getIR`, returns a loaded package's graph as MLIR text. On AFM it yields only placement operations and 108 calls into the ANE program, but with exact entry signatures (for example, the two 64×128 RoPE tables arrive as runtime inputs). On Xcode's code-completion model (CodeLM v3, a GPU model built with the same modelling library, *tamm*) it yields the complete source-level graph. CodeLM became our healthy reference model (Section 7).

4. Architecture

4.1. Dimensions.

4.2. The layer: a post-normalised "x/y" residual.

Each layer carries four learned RMSNorm gains $g_0, \ldots, g_3 \in \mathbb{R}^{1536}$. Unlike a pre-norm transformer, both residual additions add *normalised*

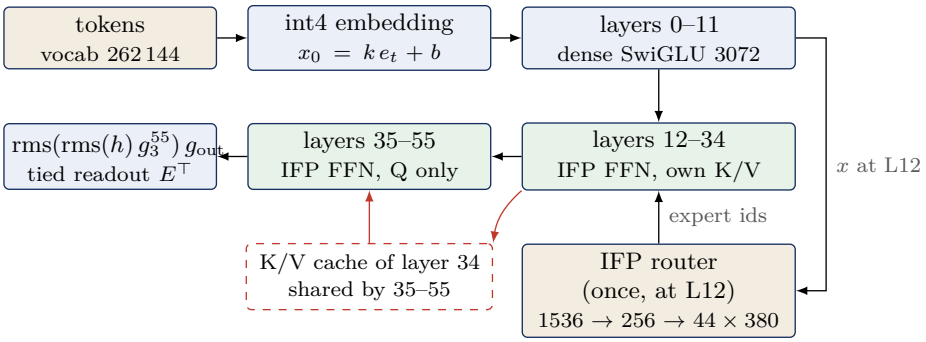


Figure 1. The whole model. The router reads the normalised input of layer 12 and emits routing logits for all 44 sparse layers at once. Layers 35–55 compute queries only and attend to the K/V cache written by layer 34.

Quantity	Value	Evidence
layers	56 (12 dense, 23 sparse own-KV, 21 sparse KV-reuse)	chunk-set plan; task stream
hidden / head dim	1536 / 128	conv shapes
heads	16 query, 8 key/value; query head h uses KV head $\lfloor h/2 \rfloor$	attention task addressing; same-token probe
dense FFN	SwiGLU, width 3072 (layers 0–11)	conv shapes; SiLU lookup table on the gate
experts	380 per sparse layer, width 256; 0–22 always on; 32–379 routable	router mask; static kernels = experts 0–22
RoPE	NEOX half-split, $\theta = 5 \times 10^5$, 128 dims	odix cos table (row 0 = 1, halves equal)
norm ε	1.0014×10^{-5}	ANE register; identical in CodeLM
attention scale	$1/\sqrt{128}$	post-scale = $(1/s_K)/\sqrt{128}$ in the hardware; CodeLM `sdpa`
vocabulary	262 144 text tokens (Gemma-3 family layout), BOS 2, end-of-turn 106	tokenizer asset
context	8192	asset metadata

Table 4. Architecture parameters and where each was read.

values:

$$
\begin{aligned}
x_L &= \mathrm{rms}(h_{L-1}) \odot g_3^{L-1} \qquad \text{(layer 0: } x_0 = k\,e_t + b,\ \text{no norm)} \\
h'_L &= x_L + \mathrm{rms}\big(\mathrm{Attn}_L(x_L)\big) \odot g_0^L \\
y_L &= \mathrm{rms}(h'_L) \odot g_1^L \\
h_L &= y_L + \mathrm{rms}\big(\mathrm{FFN}_L(y_L)\big) \odot g_2^L
\end{aligned}
$$

with $\mathrm{rms}(v) = v/\sqrt{\mathrm{mean}(v^2) + \varepsilon}$, gains used literally (no $1 + g$), and a final output $\mathrm{rms}(\mathrm{rms}(h_{55}) \odot g_3^{55}) \odot g_{\mathrm{out}}$ multiplied by the tied embedding. The residual wiring was read from the DMA pattern of the ANE program: the normalised attention input is written to DRAM scratch and read back by the attention residual add; the FFN residual add reads the normalised FFN input. Only this wiring reproduces Apple's outputs (standard residual: 0% top-1).

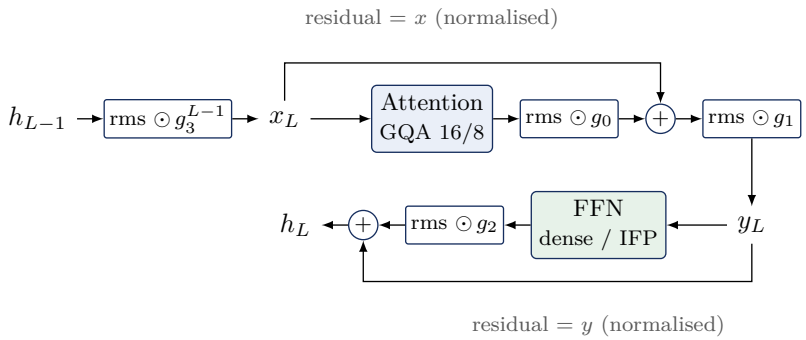


Figure 2. One layer. Both residual additions add normalised streams; g_0 and g_2 are zero-centred (about 30% negative entries), g_1 and g_3 small and positive. Zero-centred literal gains are normal in Apple's modelling library: a healthy Apple model (CodeLM) shows the same pattern.

4.3. **Attention.** Attention is standard grouped-query attention with two Apple-specific details: the query/key normalisation sits after RoPE, and the key/value cache is int8 (Figure 3).

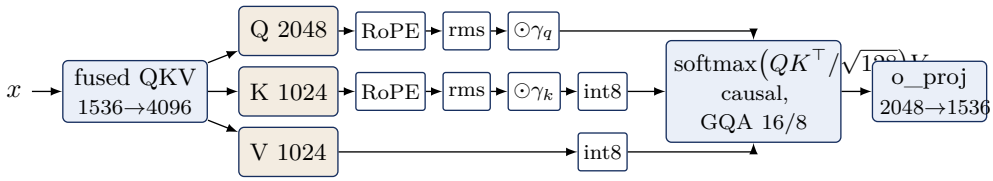


Figure 3. Attention as executed on the ANE. RoPE comes before the per-head RMS norm and its 128-entry gain (the same order appears in CodeLM's source graph). Keys and values are stored in an int8 cache with per-layer calibrated scales; layer 34 has no key gain. Layers 35–55 compute only Q and read layer 34's cache.

4.4. **The Instruction-Following-Pruned feed-forward network.** Sparse layers add two SwiGLU blocks without any gating weights:

$$\mathrm{FFN}(y) = \underbrace{\sum_{e=0}^{22} \mathrm{SwiGLU}_e(y)}_{\text{always on}} + \underbrace{\sum_{e\in\mathcal{S}(y)} \mathrm{SwiGLU}_e(y)}_{|\mathcal{S}|=23 \text{ routed}}, \qquad \mathrm{SwiGLU}_e(y) = W_e^{\mathrm{down}}\big(\mathrm{SiLU}(W_e^{\mathrm{gate}}y)\odot W_e^{\mathrm{up}}y\big),$$

each expert being 256 wide. The ANE program runs the always-on experts from static kernels and the routed experts from a mutable kernel patched at runtime from `ifp_rasterized_weights.bin`. Both halves matter: the always-on experts alone reach 62.9% top-1, the routed experts alone 1.4%, both together 87.5% (measured with the earlier template).

When experts are chosen. Apple's runtime does not route per token. It runs the dense layers over the whole prompt, takes the layer-12 input of the *last* prompt token, runs the router once, and uses that expert set for every prompt token and for generation; it re-picks from the current token only at absolute positions that are multiples of 32. Section 8 shows how this was read out of Apple's inference framework.

5. How the weights are stored

Exact GGUF encoding. llama.cpp's Q2_K stores, per 256 weights, sixteen 4-bit sub-block scales sc and mins m, a super-scale d and super-min $d_{\min}$, and evaluates $w = d\,sc\,c - d_{\min}\,m$. Choosing

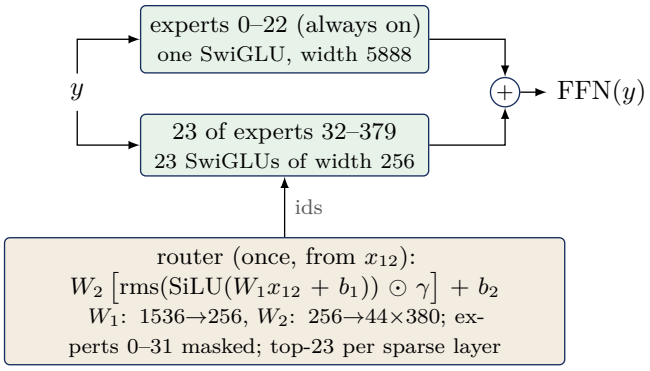


Figure 4. The IFP feed-forward network of a sparse layer and the router that feeds it. The sum is unweighted: the router only selects.

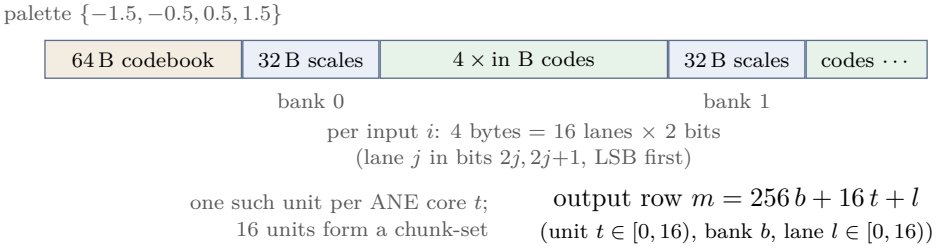


Figure 5. The 2-bit weight unit. A projection is split across the 16 ANE units bank-cyclically; a chunk with more banks holds the lower bank slots. The value of a weight is $w = s_m\,(c - 1.5)$ with $c \in \{0, 1, 2, 3\}$ and s_m the fp16 row scale. Experts in `ifp_rasterized_weights.bin` use the same codes with a simpler order: code $p \mapsto$ (row $16\lfloor p/(16\,\mathrm{in})\rfloor + p \bmod 16$, input $\lfloor p/16\rfloor \bmod \mathrm{in}$).

$sc = 2$, $m = 3$ and $d = d_{\min} = s/2$ gives

$$w = \tfrac{s}{2} \cdot 2c - \tfrac{s}{2} \cdot 3 = s\,(c - 1.5),$$

and halving an fp16 number is exact (barring underflow, which never occurs here). The int4 embedding ($w = s_t\,q$, $q \in [-7, 7]$, per-token fp16 scale s_t) maps exactly onto Q4_0 with $d = s_t$ and nibble $q + 8$. Dequantising the GGUF with gguf-py reproduces Apple's weights with maximum absolute difference 0.

Layout facts and how they were ground-truthed. We compiled known weights through Apple's own ANE compiler (`mil_to_hwx -a h16`) and decoded the result: the output axis is the bank-cyclic stripe above, the input axis is natural, the codebook and row scales are as in Figure 5, and constants (gains, biases) are stored in natural order. A calibrated copy-circuit probe (Section 7) independently confirms the stripe, the head order and the V/O pairing: every alternative layout scores at chance.

6. The norm-gain table, and the bug that hid the model

All gains live in one contiguous fp16 table inside `__TEXT,__const` of `binary_0.hwx`:

Finding 1. The earlier parse located the table by statistical signatures and accepted a start two bytes early. Every RMSNorm therefore multiplied channel r by the gain of channel $r-1$. Because g_0 and g_2 are zero-centred, this flipped the sign of about a third of every residual write at every layer, and the assembled model predicted nothing (0% top-1, no induction).

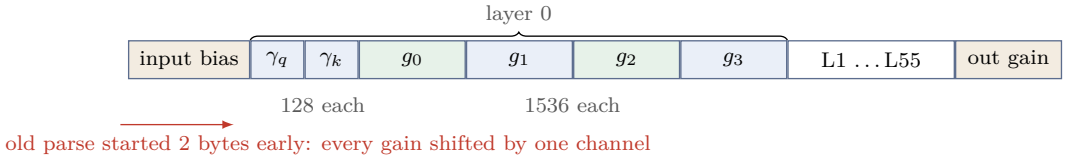


Figure 6. The gain table. Layers 0–33 carry γ_q and γ_k; layers 34–55 only γ_q. Every vector length is a multiple of 64 bytes, so a correct start is 64-byte aligned.

How it was found. Nothing looked wrong in isolation: each tensor was correctly laid out and correctly paired. The breakthrough was calibrating probes on a healthy Apple model and then comparing *reader–reader* with *reader–writer* relations. In AFM, query–key and gate–up relations were as healthy as in CodeLM, but value–output (copy heads) and up–down (FFN neuron coupling) relations were weak. In a trained network, the sign of the coupling between up-row i and down-column i agrees with the sign of the gain that scales the write. In AFM the *number* of negatively coupled channels equalled the number of negative entries of g_2 in every layer—while their *positions* disagreed (Figure 7, left). Matching each channel's 11-layer sign signature against the table gave a constant offset of one element. Read aligned, the agreement rises to 98–99.9% (right) and the model works: 66% top-1 against Apple's outputs on the first attempt, rising to 87.5% with the llama.cpp port and exact tokenization.

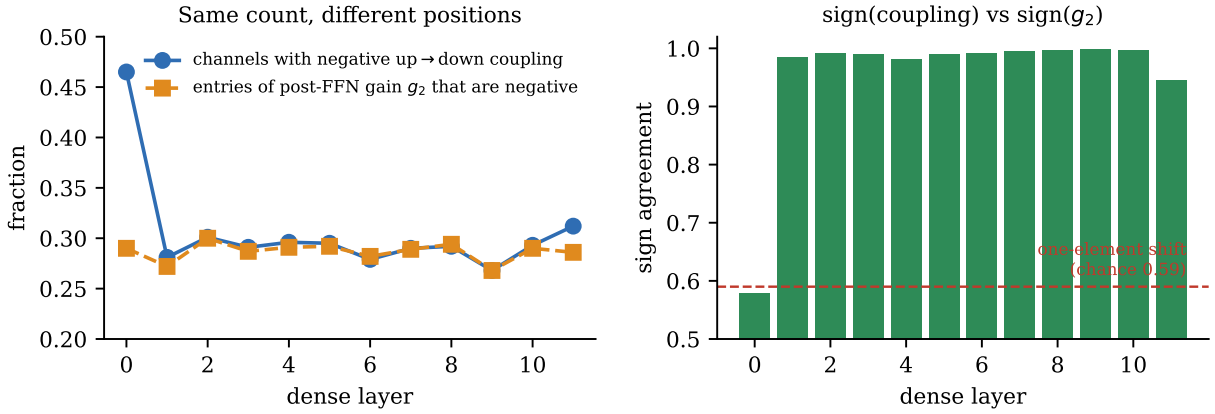


Figure 7. Left: per layer, the fraction of residual channels with negative up→down coupling (from the weights) and the fraction of negative entries in the decoded post-FFN gain g_2—equal counts. Right: per-channel sign agreement after reading the table aligned; a one-element shift gives chance level. Layer 0 is the exception because its input is the raw embedding.

7. Validation instruments and their calibration

The oracle. `afm -t 0` (a small Swift wrapper around the Foundation Models API) produces Apple's greedy answers; at temperature zero, Apple's top-1 token at every answer position is the next token, so teacher-forced top-1 agreement is an absolute measure. We use 40 prompts (1127 answer tokens). The API exposes no probabilities.

A healthy reference: CodeLM v3. Xcode's on-device code model ships as a GPU MPSGraph package. Its MLIR (via `-getIR`) is a complete, readable tamm transformer—pre-norm, standard residual, RoPE then QK-norm, SwiGLU, KV reuse from layer 34, int4 weights with group-32 scales—with the same backbone shape as Apple's other 3B-class models (56 layers, 35 KV-owning, 2048 wide, 16/2 heads). We ran it on the GPU from our own process through `MPSGraphExecutable` (100%

induction accuracy) and reproduced its logits in NumPy exactly (correlation 1.00000, 100% argmax agreement). Its tokenizer and embeddings are unrelated to v7-3b's, so it cannot donate weights—but it calibrates instruments.

What the calibration showed.

- *Zero-centred literal gains are normal* in tamm (CodeLM's pre-attention gain: mean +0.003, 46% negative).
- *The embedding–weight "alignment ratio" used for months is not diagnostic*: a healthy model scores the same (1.0–1.4).
- *The OV copy probe is diagnostic*: CodeLM has several heads that map a token to itself at rank 0–16, even after naive 2-bit rounding; decoded AFM had none below rank 1000 (Figure 8). Every alternative V/O layout scored at chance, which ruled out layout errors and pointed at the gains.
- *The same-token QK probe* found AFM's query–key circuits healthy (rank-0 heads) and confirmed the head pairing $h \mapsto \lfloor h/2 \rfloor$.

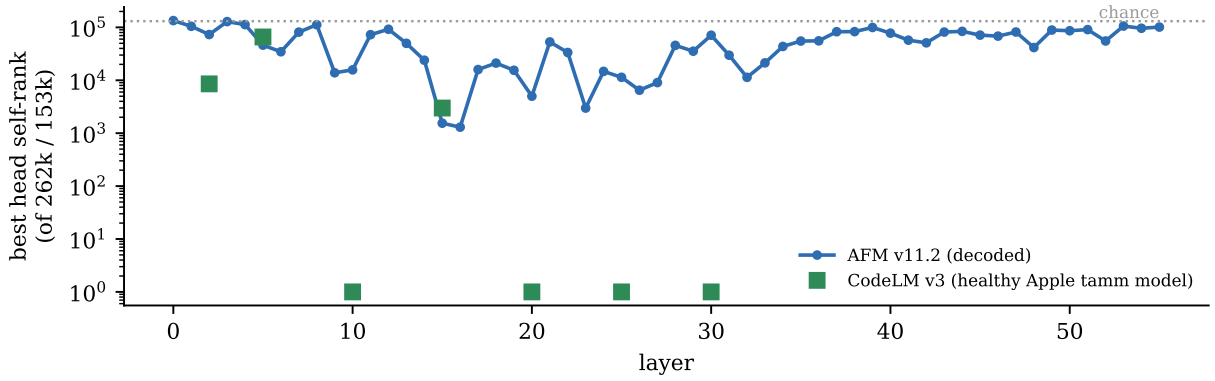


Figure 8. Copy-circuit probe: for each layer, the best attention head's median rank of the input token in $E\,W_O W_V\,e_t$. Healthy CodeLM has rank-0 heads; AFM's raw $W_O W_V$ peaks at weak copying because in AFM the write passes through the signed gain g_0, whose positions were scrambled by the table shift.

Apple's int8 calibration as a ground truth. The ANE program stores per-layer scales for its int8 K/V cache. Apple chose them from real activations, so they bound the true magnitudes. With the corrected gains, the reconstruction's values stay inside Apple's ranges, and its keys stay inside up to the 99.9th percentile; only the most extreme 0.1% of keys would clip (Figure 9). This is an independent check of the internal scales. (Figure 9 was made with the v1.0 forward; with the correct layer-0 input the layer-0 values use only part of Apple's range, so Apple's layer-0 scale is not a tight bound.)

8. What Apple's runtime does: the prompt and the expert schedule

Two things the model files do not contain decide what the model sees and which experts it uses: the prompt template (the metadata only names it, `afm_cider_v2`) and the rule for choosing experts. Both live in code, in two private frameworks of the dyld shared cache: `TokenGenerationCore` renders and tokenizes prompts, `TokenGenerationInference` drives the model. `dyld_info -disassemble` prints both with Swift symbol names. One detail matters for reading them: Swift stores string literals of up to 15 bytes inside the instruction stream (`mov`/`movk` immediates), so `<start_of_turn>` and friends never show up in a strings dump.

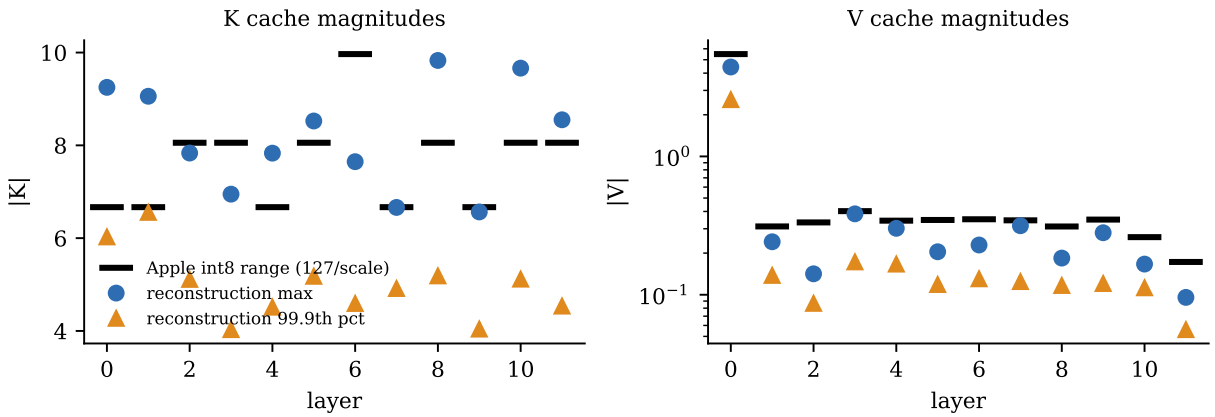


Figure 9. Reconstructed key and value magnitudes (dense layers, one prompt) against Apple's calibrated int8 ranges. Layer 0's values are large because its input is the un-normalised embedding.

8.1. **The prompt template.** `PromptPreprocessingTemplateAfmCiderV2Convention` takes its special-token table from the older Cider v1 convention and overrides only audio and reasoning entries. Decoding the immediates gives turn start `<start_of_turn>`, turn end `<end_of_turn>`, and roles `system\n`, `developer\n`, `user\n`, `model\n`; the "thought budget: none" and "developer static instructions" entries are empty. Its `cacheableSystemPrefix` concatenates turnStart, the system role with Apple's default instructions, the thought budget, turnEnd, a newline, then turnStart and the developer role. A FoundationModels conversation therefore renders as

<bos><start_of_turn>system\n*default*<end_of_turn>\n
<start_of_turn>developer\n*instructions*\n<end_of_turn>\n
<start_of_turn>user\n*prompt*<end_of_turn>\n<start_of_turn>model\n

where *default* is the `systemInstructions.default` text of the adapter metadata ("A conversation between a user and a helpful assistant. You are a foundation model developed by Apple. . . . ") and the developer turn holds the app's instructions. The public API verifies every detail: `SystemLanguageModel.tokenCount(for:)` accepts transcript entries and returns the length of the rendered prompt, which equals the length the inference service logs ("asked to run 71 tokens") for the same conversation. Table 5 compares it with our rendering. Our earlier template folded everything into one system turn and omitted the instructions our reference CLI passes; with the exact template, teacher-forced agreement rose from 87.5% to 90.5% and the first answer token from 30 to 38 of 40.

Transcript (I = instructions, P = prompt, R = response)	Apple `tokenCount`	our template
P only	16	16
I (empty), P	62	62
I only	55	55
I, P	71	71
I, P, R, P	86	86

Table 5. Prompt lengths. A session without instructions still carries the system turn and an empty developer turn (62 tokens, seen in the live log); each instruction segment ends with a newline; earlier answers are closed with `<end_of_turn>\n`.

8.2. **The expert schedule.** The inference log of a single request already shows the shape of the algorithm: a "prompt expert compute function" (`lora_48_ifp3_r48_prompt_opt_dense_only_256_64`, the dense layers only) runs over all prompt tokens, then "Picking IFP experts at position=70 isPromptPick=true" for a 71-token prompt, then the sparse layers run over the *whole* prompt again with the picked experts. The dense-only function returns a `1536×64` fp16 tensor per 64-token chunk, one column per token. In `TokenGenerationInference`, `computePromptExpertState` builds the router input with `IFPDenseLayerOutput.makeExpertsTensor(tokenIndex:)`, whose loop copies the 1536 values of a single column into a 1536×1 tensor—no mean, no accumulation. The router (`project_experts`, the network of Figure 4) then yields the top-23 experts per sparse layer. During generation, `repickExpertsAtBoundary` calls the same picker, and `IFPExpertPicker` rejects any intermediate pick whose position is not a multiple of `expert_selection_frequency` (an `sdiv`/`msub` remainder test against 32). With prompt length P, token t therefore uses the experts chosen from x_{12} at

$$s(t) = \max(P - 1,\ 32\lfloor t/32 \rfloor),$$

so every prompt token uses the last prompt token's choice. Averaging the prompt instead is clearly wrong (79.3% teacher-forced), while the single-token rule beats per-token routing (Section 10).
An outside check of the router. Each pick also logs `CacheExpertExpertCache: ... replaced=`n, the number of (layer, expert) choices that changed. We ran six long generations on Apple's model, collected its 47 re-picks, replayed the same token sequences through our model and counted how many of our choices change at the same positions. With the single-token rule on the layer-12 attention input the two series agree closely (Figure 10: mean 26.7 versus 27.1 experts, correlation 0.84); prefix means correlate at 0.21–0.53 and the FFN-input variant changes three times too many. This validates the router input, the router network and the rule independently of any answer text.

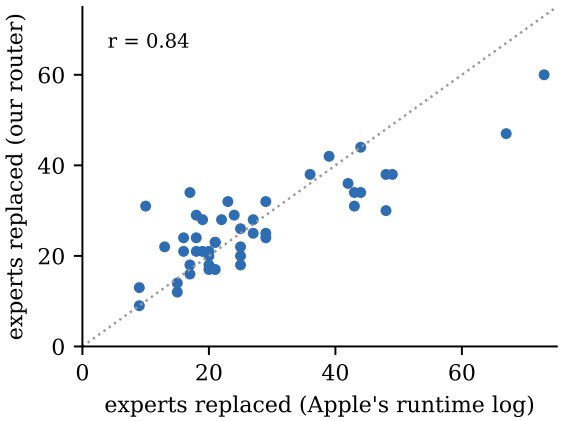


Figure 10. Experts replaced at each re-pick during generation: Apple's runtime log versus our router replaying the same tokens (47 re-picks in six generations).

9. The llama.cpp implementation

The fork adds one architecture, `afm`, in `src/models/afm.cpp` (about 180 lines), plus three supporting changes.

- **Graph.** Layer 0 takes the dequantised embedding row as is (no multiplier) and, for generated tokens only, adds the input bias (Section 10); each later layer normalises the previous output with g_3; queries and keys use `rms_norm`, NEOX RoPE, then a gain multiply; the residuals follow Figure 2. Sparse layers run the always-on experts as a "shared expert" SwiGLU of width 5888 and the routed experts with `mul_mat_id`, summed by one row reduction with no

weights. The router network lives on layer 12 and its logits are sliced per layer. The output applies the double norm and the tied embedding.

- **KV reuse.** Layers 35–55 call attention with null K/V, and the cache maps them to layer 34. This needed a two-line fix in the non-sliding-window attention path, which assumed K/V were always present.
- **Metal kernels.** Metal's batched expert matmul is instantiated for specific experts-per-token counts; we added 14 and 23 (ifp2, ifp3). CUDA uses a generic path for such counts, and Vulkan supports Q2_K expert matmuls, so no change is needed there.
- **Apple's expert schedule.** The router's logits for each sequence persist in a small device buffer (64 sequence slots) owned by the model. A custom graph input computes, per micro-batch on the host, which logits every token uses: a run of several tokens of one sequence is a prompt chunk and takes its last token's logits, which are stored; a single decode token re-picks only when its position is a multiple of 32 and otherwise reads the stored choice. Because the indices are data rather than graph structure, llama.cpp's graph reuse and multi-sequence batching stay correct. `AFM_ROUTE_TOKEN=1` restores per-token routing.
- **The embedding-scale pitfall.** llama.cpp's generic `build_inp_embd()` multiplies by the GGUF's `embedding_scale` (a Granite feature). The first release also multiplied in the AFM graph, so a GGUF carrying our fitted 10 ran at 100. The loader now ignores the key, so old GGUFs run correctly.
- **int8 KV emulation.** The GGUF carries Apple's per-layer K/V scales; keys and values are rounded and saturated to int8 before caching. Rounding uses the fp32 magic constant $1.5 \cdot 2^{23}$, because `ggml_round` misbehaved on Metal.

On an M4 the model processes prompts at about 150 tokens/s and generates 12–21 tokens/s. The patch applies to upstream llama.cpp (commit `41ea261`); the repository's workflow builds Windows (CPU, Vulkan, CUDA), Linux (CPU, Vulkan) and macOS (Metal) binaries from it.

10. Fidelity

The reference is Apple's own model, queried through the public FoundationModels API at temperature 0 with the instructions "You are a helpful, concise assistant." (`tools/afm.swift`) on 40 prompts. We measure two things: *teacher-forced* top-1 agreement (Apple's answer is fed in and we count how often our most likely next token is Apple's; 1127 answer tokens) and *free-running* agreement (our own greedy answer compared with Apple's, character for character).

Routing (exact template)	teacher-forced	first token	identical answers
Apple's schedule (pick at last prompt token, re-pick every 32)	**96.10%**	37/40	17/40
per token	93.79%	36/40	12/40
v1.0: Apple's schedule, embedding ×100, bias on every token	91.75%	35/40	7/40
v1.0: per token	90.51%	38/40	11/40

Table 6. Routing rules. "Identical answers" = our free-running greedy answer equals Apple's exactly (up to 200 tokens); one different early token sends the rest of an answer elsewhere, so this measure is dominated by the first few tokens.

The layer-0 input: no multiplier, and a bias only for generated tokens. Earlier drafts fitted an input multiplier k and found the score flat for $k = 8$–14. That scan was invalid: llama.cpp's generic embedding code already applied the GGUF's `embedding_scale`, and the AFM graph applied it again, so the scan really covered $k = 80$–140. Dumping llama.cpp's intermediate tensors and

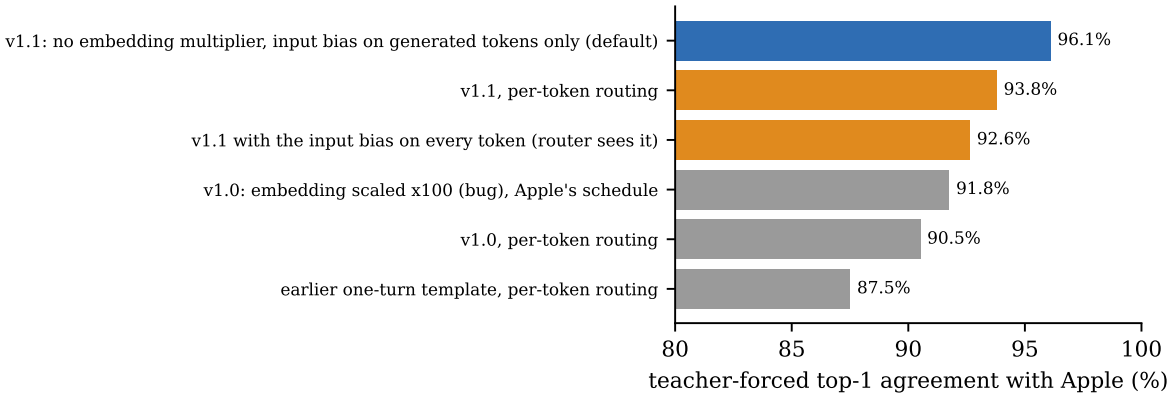


Figure 11. Teacher-forced top-1 agreement with Apple's greedy answers (40 prompts, 1127 tokens). Blue: the shipped default. The template fix is worth three points, the layer-0 fixes (no embedding multiplier, input bias on generated tokens only) four more, and Apple's routing schedule beats per-token routing.

Ablation (earlier 10-prompt set, earlier template)	top-1
always-on + 23 routed experts	85.6%
always-on experts only	62.9%
routed experts only	1.4%
16 instead of 23 routed / 14 routed	83.5% / 82.1%
standard (pre-norm) residual instead of x/y	0%

Table 7. Ablations: both expert groups are needed, the routed count matters, and the residual form is not negotiable.

comparing them with an independent NumPy forward exposed a factor of exactly 10 at the layer-0 input. Scanning the true multiplier with the expert picks held fixed gives a sharp optimum at $k = 1$ (mean log-loss of Apple's tokens 0.140, against 0.143 at 0.75 and 1.25 and 0.240 at 100). This matches what Apple's files say: the gather has no multiply.

With $k = 1$ the body of the network and the router disagreed. The body wanted the input bias added, but the router's picks were best when the bias was negligible. We found this by computing the picks from a forward at a larger k, or with the bias removed; a version that changed only the layer-0 attention balance did not help (78%). Apple's ANE program has two procedure families. The prompt procedures (`prompt_opt`) add a zero constant at layer 0. The extend procedures, which compute generated tokens, add the bias vector. Applying the bias to generated tokens only is one consistent model for both parts. It reaches 95.9% with the picks held fixed and 96.1% in the shipped runtime, which gives the bias to single-token decode steps and not to prompt chunks. Flipping the bias's sign costs two points, so the vector is real; it simply belongs to one procedure family.

The tokenizer. Apple's `SystemLanguageModel.tokenCount` gives Apple's own token count. Our first tokenizer (and the research encoder) matched it on 73 of 81 texts; it had byte-fallback tokens labelled as normal text and control tokens mislabelled. The converter's tokenizer matches on 81/81 (and on 16/16 texts built from tabs, runs of spaces and indented code) and lifted fidelity from 86.0% to 87.5%. Apple's tokenizer file marks all user-defined pieces with one type code; mapped naively, llama.cpp prints space runs as the escape character U+2581 instead of spaces and drops tab runs entirely, so the converter types space runs as normal text and tab runs as user-defined.

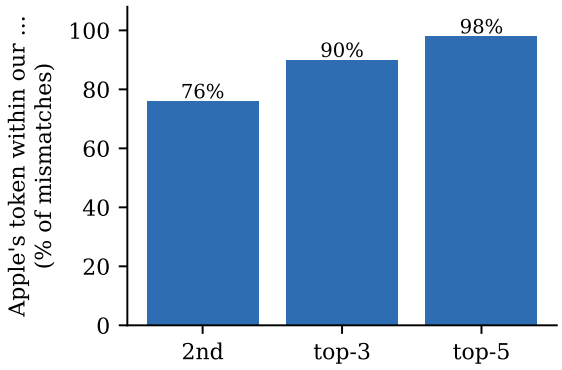


Figure 12. Where the 93 disagreements of the v1.0 configuration sat: Apple's token is our second choice in 76%, in our top three in 90% and in our top five in 98%; the median logit gap to it is 1.15, and 45% of the misses are within one logit.

11. The converter

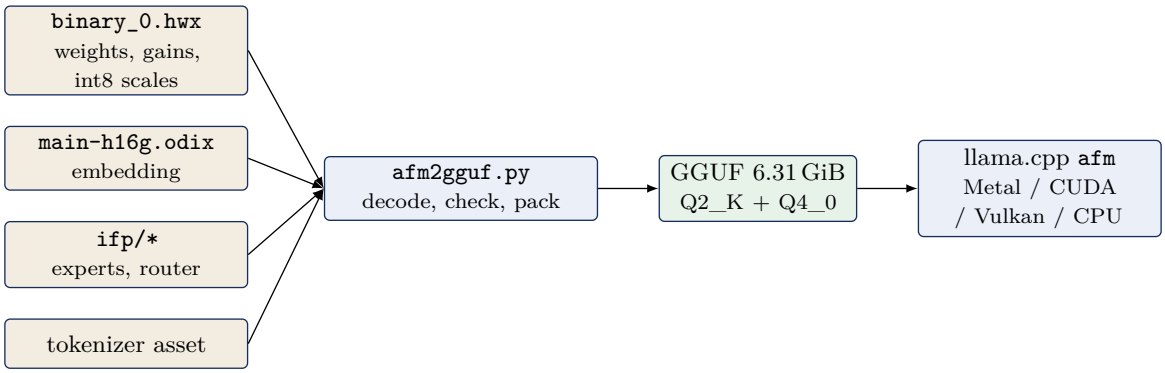


Figure 13. Conversion runs on the Mac that holds the assets (2–3 minutes); the GGUF then runs anywhere.

`afm2gguf.py` needs only NumPy and gguf-py and contains no Apple data. It finds assets by their `AssetSpecifier`, decodes everything directly, and refuses to write if a check fails: palette and container layout, 3504 units in 219 chunk-sets, the router mask, the tokenizer layout, the embedding scale array (chosen by row-norm consistency) and—decisively—the gain-table alignment, chosen by the FFN coupling-sign agreement (0.99 for the true start, 0.59 for the one-vector-early candidate that passes every shape test). Its output is byte-identical, tensor by tensor, to the validated export.

12. What did not work

Recorded so that nobody repeats them:

- Running Apple's backbone on the ANE from our own process: the kernel refuses LLM base-model requests from non-platform binaries, even with AMFI disabled.
- Ten-plus forward variants before the gain fix: Gemma-style $1+g$ gains, standard residual, swapped or negated RoPE tables, embedding scales, alternative row stripes, GQA maps, within-vector gain permutations, per-row scale permutations—all 0% top-1, because the defect was elsewhere.
- LoRA: the base model's adapters are untrained, and the Foundation Models API's generic adapter for v11.2 is empty (its signature is the SHA-256 of the empty string).

- Weight statistics such as "alignment ratios" and residual-collapse measures: uncalibrated, and on a healthy model indistinguishable from AFM.
- Pooled expert routing rules (mean of inputs, of logits, of probabilities; prompt or user turn; prefix mean, vote counts): all below per-token routing, and all wrong—Apple routes from a single token (Section 8). They had looked plausible only because our template was wrong.
- Routing from the layer-12 FFN input instead of its attention input: 89.4% teacher-forced, and three times too many experts changed per re-pick (109 versus Apple's 36).
- Reading Apple's between-prompt expert changes from the cache log: those counts (361–556 of 1012) include cache eviction while the service is idle and do not measure pick differences; only re-picks within one generation do.

13. Toward 100%

The prompt template, the expert schedule and the layer-0 input are now Apple's own. 44 of 1127 answer tokens still differ. Apple's token is almost always our second or third choice. Earlier drafts called the remaining gap a numerical noise floor; the layer-0 bug shows that this was premature. What we measured directly:

(1) **The ANE's matrix multiplies are accurate.** We ran our own CoreML programs on the Neural Engine, confirming the device with Core ML's compute plan. Sums accumulate at high precision and only the fp16 output is rounded (relative error 2.8×10^{-4}, the same as fp32 accumulation with an fp16 result). The low-precision accumulation one first sees comes from Core ML's fp16 CPU path, not the ANE.

(2) **Its SiLU is a 33-knot piecewise-linear table** (knots every 0.5 on $[-8, 8]$, fitted values, for example 0.2964 at $x = 0.5$ where SiLU is 0.3112). The same 128-byte table appears 2113 times in AFM's ANE program, once per FFN task, and it is the program's only activation table. We tabulated it for all 65 536 fp16 inputs and can emulate it exactly. Doing so changes 20 of 1127 predictions (10 fixed, 8 broken in the v1.0 setup), so it is not where the gap lies. Its softmax, exp and rsqrt are also approximate (0.3–0.4% relative error).

(3) **Apple's speculative decoding is exact.** Every FM-API adapter sets `draft_soft_match_tolerance` to 0 and turns rejection sampling off, so the draft model never changes a greedy token. The FM-API adapter is the declared-empty adapter (signature = SHA-256 of the empty string), and there is no repetition penalty.

(4) **The router is robust to arithmetic.** Computing it with fp16 accumulation moves 0.01 of 23 experts per layer-pick, and random noise strong enough to swap a third of an expert per layer changes 2% of predictions without moving the agreement.

Every weight, gain, bias, the router, the tokenizer, the template and the routing rule come from Apple's files and code; nothing was trained or fine-tuned.

Path noise. Two mathematically identical fp32 implementations of the same computation disagree on 0.1–2.4% of next-token choices: fp16-rounded activations against fp32, CPU against Metal, llama.cpp's RoPE kernel against an elementwise one, or one batch against a prompt batch followed by single steps. The 2-bit weights and the int8 KV cache amplify rounding, so a percent or so of residual disagreement is expected even from an exact model.

What the misses were in v1.0. With Apple's schedule, 74 of the 93 v1.0 misses were also missed with per-token routing, so they did not depend on routing. At 89 of the 93 our choice was an ordinary text token, so Apple's output masking is not involved. Table 8 lists the numerical experiments, run on the v1.0 configuration.

14. Responsible use

The model's weights belong to Apple. We publish only our own code (the llama.cpp architecture patch and the converter, at `https://github.com/yu314-coder/afm-ifp-teardown`), which

Variant of the default (teacher-forced, 1127 tokens)	top-1
v1.0 default: Apple's schedule, int8 KV emulation, embedding ×100	91.75%
int8 KV emulation off	90.77%
no re-pick during generation	90.95%
re-pick takes effect one token later	91.39%
activations rounded to fp16 (residual / + router input / + every sublayer output)	91.39%
RoPE angles computed in fp16 / + fp16 cos and sin	91.75% / 91.57%
attention scale ×0.8 / ×0.9 / ×1.1 / ×1.25 (of $1/\sqrt{128}$)	87.8 / 90.1 / 89.9 / 89.9%
tanh soft-cap of attention scores at 50 / 30 / 20 (Gemma-style)	91.4 / 90.4 / 89.5%
our own fp32 RoPE instead of llama.cpp's (same mathematics)	91.84%

Table 8. Numerical and schedule experiments. Emulating Apple's int8 KV cache and re-picking every 32 tokens are both worth close to a point, the attention scale peaks sharply at $1/\sqrt{128}$, and there is no attention soft-cap; storing activations or RoPE angles in fp16 changes nothing beyond the noise floor.

rebuilds the model from the assets already on a user's Mac; we publish no weights, tokenizer data or decoded assets, and have asked Apple whether redistribution would be permitted. The project is unofficial and not affiliated with or endorsed by Apple.

Appendix A. GGUF layout

Tensor	Shape (rows × cols)	Type
`token_embd`	262144 × 1536	Q4_0
`input_bias`, `output_norm`, `output_norm_2`	1536	F32
`blk.L.attn_q`	2048 × 1536	Q2_K
`blk.L.attn_k`, `attn_v` ($L \le 34$)	1024 × 1536	Q2_K
`blk.L.attn_output`	1536 × 2048	Q2_K
`blk.L.attn_q_norm`, `attn_k_norm` ($L < 34$)	128	F32
`blk.L.{post_attention,ffn,post_ffw,layer_output}_norm`	1536	F32
`blk.L.ffn_{gate,up}` / `ffn_down` ($L < 12$)	3072 × 1536 / 1536 × 3072	Q2_K
`blk.L.ffn_{gate,up}_shexp` / `ffn_down_shexp` ($L \ge 12$)	5888 × 1536 / 1536 × 5888	Q2_K
`blk.L.ffn_{gate,up}_exps` / `ffn_down_exps`	348 × 256 × 1536 / 348 × 1536 × 256	Q2_K
`blk.12.ifp_router_fc1` (+bias), `_norm`, `_fc2` (+bias)	256 × 1536, 256, 15312 × 256	F32

Table 9. 805 tensors, 6.31 GiB. Metadata keys under `afm.*`: block count 56, embedding length 1536, head counts 16/8, key length 128, expert count 348, experts used 23, shared expert width 5888, leading dense blocks 12, KV-reuse first layer 35, RoPE base 5×10^5, RMS ε, embedding scale, per-layer K/V int8 scales, and a Jinja chat template.

Email address: euler.yu@gmail.com